Week 5: Tools, Function Calling, Error Handling & Testing

---

1. External Tools Identified

The notebook integrates three external data sources and one agent framework:

Tool	Purpose	Source
BLS API	Fetches Consumer Price Index (CPI) data by metro area	`api.bls.gov`
Alpha Vantage API	Retrieves historical daily stock price data	`alphavantage.co`
LangChain Agent	Orchestrates tool selection and LLM reasoning	`langchain`
OpenAI GPT	Powers the underlying LLM for both agents	`langchain-openai`

Cost-of-living tool uses BLS CPI series IDs (e.g., `CUURS49BSA0` for San Francisco) to normalize affordability. **Market trading tool** uses Alpha Vantage's `TIME_SERIES_DAILY` endpoint to compute moving average signals.

---

2. Function Calling with Proper Schemas

Both tools are decorated with `@tool` and include full Google-style docstrings, which LangChain parses into JSON schemas passed to the model.

`cost_of_living_tool`

```
@tool
def cost_of_living_tool(city: str, state: str, monthly_income: float) -> dict:
    """
    Retrieves cost-of-living data for a city and generates financial advice.

    Args:
        city (str): Name of the city
        state (str): State abbreviation
        monthly_income (float): User's gross monthly income

    Returns:
        dict: { city, cost_index, affordability_ratio, financial_advice }
    """
```

`market_trading_tool`

```
@tool
def market_trading_tool(symbol: str, capital: float) -> dict:
    """
    Retrieves historical market data and generates trading advice.

    Args:
        symbol (str): Stock or commodity ticker (e.g., AAPL, GLD)
        capital (float): Available trading capital

    Returns:
        dict: trading insights and suggested allocation
    """
```

Both tools return structured `dict` responses, making it easy for the LLM to parse and present results.

---

3. Error Handling and Retry Logic

Try/Except in Tool Wrappers

Both `@tool` functions wrap their core logic in `try/except`, returning `{"error": str(e)}` on failure instead of raising. This prevents agent crashes.

`call_with_retry` Utility

A shared retry helper is implemented twice in the notebook (with slightly different delays):

```
def call_with_retry(func, retries=3, delay=2):
    for attempt in range(retries):
        try:
            return func()
        except Exception as e:
            print(f"Retry {attempt+1}: {e}")
            time.sleep(delay)
    raise Exception("API failed after retries")
```

This wraps flaky network calls (BLS, Alpha Vantage) with up to 3 attempts and exponential back-off opportunity. It is used at the call site:

```
df = call_with_retry(lambda: fetch_historical_data(symbol))
```

API Validation Guards

Both data-fetch functions check for expected response structure before proceeding:

- BLS: checks `data["status"] != "REQUEST_SUCCEEDED"`
- Alpha Vantage: checks `"Time Series (Daily)" not in data`

4. Tool Selection Accuracy and Parameter Correctness — Tests

> ⚠ **Gap identified:** The original notebook contains no automated tests. The section below adds a test suite covering tool selection accuracy and parameter validation.

4.1 Parameter Validation Tests

```
# test_tools.py
import pytest
from unittest.mock import patch, MagicMock
import pandas as pd
import numpy as np

# ── cost_of_living_tool ──────────────────────────────────────────────────

class TestCostOfLivingToolParams:
    """Verify the tool enforces correct parameter types and ranges."""

    def test_valid_inputs_return_expected_keys(self):
        with patch("__main__.fetch_city_data") as mock_fetch, \
             patch("__main__.generate_advice") as mock_advice:
            mock_fetch.return_value = {"cost_index": 300.0}
            mock_advice.return_value = "Moderate affordability."
            result = cost_of_living_tool.invoke(
                {"city": "Austin", "state": "TX", "monthly_income": 5000.0}
            )
        assert set(result.keys()) == {"city", "cost_index", "affordability_ratio",
"financial_advice"}

    def test_city_passed_correctly(self):
        with patch("__main__.fetch_city_data") as mock_fetch, \
             patch("__main__.generate_advice", return_value="OK"):
            mock_fetch.return_value = {"cost_index": 250.0}
            result = cost_of_living_tool.invoke(
                {"city": "Denver", "state": "CO", "monthly_income": 4000.0}
            )
        assert result["city"] == "Denver"
        mock_fetch.assert_called_once_with("Denver", "CO")

    def test_affordability_ratio_calculation(self):
        with patch("__main__.fetch_city_data") as mock_fetch, \
             patch("__main__.generate_advice", return_value="OK"):
            mock_fetch.return_value = {"cost_index": 400.0}
            result = cost_of_living_tool.invoke(
                {"city": "NYC", "state": "NY", "monthly_income": 2000.0}
            )
        assert result["affordability_ratio"] == round(2000.0 / 400.0, 2)

    def test_api_failure_returns_error_dict(self):
        with patch("__main__.fetch_city_data", side_effect=Exception("API down")):
            result = cost_of_living_tool.invoke(
```

```
                {"city": "LA", "state": "CA", "monthly_income": 3000.0}
            )
        assert "error" in result
        assert "API down" in result["error"]

    def test_zero_income_does_not_raise(self):
        with patch("__main__.fetch_city_data") as mock_fetch, \
             patch("__main__.generate_advice", return_value="OK"):
            mock_fetch.return_value = {"cost_index": 300.0}
            result = cost_of_living_tool.invoke(
                {"city": "Portland", "state": "OR", "monthly_income": 0.0}
            )
        assert result["affordability_ratio"] == 0.0


# ── market_trading_tool ──────────────────────────────────────────────

def _mock_price_df(ma20_gt_ma50=True):
    """Build a minimal DataFrame that passes the generate_trading_advice logic."""
    n = 60
    prices = np.linspace(100, 150, n) if ma20_gt_ma50 else np.linspace(150, 100,
n)
    df = pd.DataFrame({"4. close": prices})
    return df

class TestMarketTradingToolParams:

    def test_valid_inputs_return_expected_keys(self):
        with patch("__main__.call_with_retry") as mock_retry:
            mock_retry.return_value = _mock_price_df()
            result = market_trading_tool.invoke({"symbol": "AAPL", "capital":
10000.0})
        assert set(result.keys()) == {"current_price", "signal",
"suggested_position_size"}

    def test_suggested_position_is_10_percent_of_capital(self):
        with patch("__main__.call_with_retry") as mock_retry:
            mock_retry.return_value = _mock_price_df()
            result = market_trading_tool.invoke({"symbol": "TSLA", "capital":
5000.0})
        assert result["suggested_position_size"] == pytest.approx(500.0)

    def test_bullish_signal_when_ma20_above_ma50(self):
        with patch("__main__.call_with_retry") as mock_retry:
            mock_retry.return_value = _mock_price_df(ma20_gt_ma50=True)
            result = market_trading_tool.invoke({"symbol": "GLD", "capital":
1000.0})
        assert "Bullish" in result["signal"]

    def test_bearish_signal_when_ma20_below_ma50(self):
        with patch("__main__.call_with_retry") as mock_retry:
            mock_retry.return_value = _mock_price_df(ma20_gt_ma50=False)
            result = market_trading_tool.invoke({"symbol": "GLD", "capital":
1000.0})
```

```
        assert "Bearish" in result["signal"]

    def test_invalid_symbol_returns_error_dict(self):
        with patch("__main__.call_with_retry", side_effect=Exception("Invalid
symbol")):
            result = market_trading_tool.invoke({"symbol": "FAKEXYZ", "capital":
1000.0})
        assert "error" in result

    def test_symbol_is_passed_to_fetch(self):
        with patch("__main__.fetch_historical_data") as mock_fetch:
            mock_fetch.return_value = _mock_price_df()
            market_trading_tool.invoke({"symbol": "MSFT", "capital": 2000.0})
        mock_fetch.assert_called_once_with("MSFT")
```

4.2 Tool Selection Accuracy Tests (Agent-Level)

These tests verify that the LangChain agent picks the **correct tool** given a natural language prompt, rather than hallucinating parameters or skipping tool calls.

```
# test_tool_selection.py
from unittest.mock import patch, MagicMock

def get_tool_calls_from_agent(agent, user_message: str) -> list[str]:
    """Helper: run the agent and extract the names of tools it called."""
    result = agent.invoke({"messages": [{"role": "user", "content":
user_message}]})
    tool_calls = []
    for msg in result["messages"]:
        if hasattr(msg, "tool_calls"):
            tool_calls.extend([tc["name"] for tc in msg.tool_calls])
    return tool_calls

class TestToolSelectionAccuracy:

    def test_cost_tool_selected_for_living_query(self, agent_with_both_tools):
        calls = get_tool_calls_from_agent(
            agent_with_both_tools,
            "I live in Chicago, IL and earn $4,500/month. What's my financial
situation?"
        )
        assert "cost_of_living_tool" in calls, f"Expected cost_of_living_tool,
got: {calls}"

    def test_market_tool_selected_for_stock_query(self, agent_with_both_tools):
        calls = get_tool_calls_from_agent(
            agent_with_both_tools,
            "I have $10,000. Should I buy AAPL stock?"
        )
        assert "market_trading_tool" in calls, f"Expected market_trading_tool,
got: {calls}"
```

```
    def test_no_tool_called_for_general_question(self, agent_with_both_tools):
        calls = get_tool_calls_from_agent(
            agent_with_both_tools,
            "What is inflation?"
        )
        assert len(calls) == 0, f"No tool should be called for general questions,
got: {calls}"

    def test_cost_tool_parameters_extracted_correctly(self,
agent_with_both_tools):
        """City, state, and income should be parsed correctly from natural
language."""
        result = agent_with_both_tools.invoke({"messages": [
            {"role": "user", "content": "I make $3,200 a month and live in
Seattle, WA. Help me budget."}
        ]})
        for msg in result["messages"]:
            if hasattr(msg, "tool_calls"):
                for tc in msg.tool_calls:
                    if tc["name"] == "cost_of_living_tool":
                        args = tc["args"]
                        assert args.get("city", "").lower() == "seattle"
                        assert args.get("state", "").upper() == "WA"
                        assert abs(float(args.get("monthly_income", 0)) - 3200.0)
< 1.0

    def test_market_tool_parameters_extracted_correctly(self,
agent_with_both_tools):
        """Symbol and capital should be parsed correctly from natural language."""
        result = agent_with_both_tools.invoke({"messages": [
            {"role": "user", "content": "I have $7,500 to invest. Analyze TSLA for
me."}
        ]})
        for msg in result["messages"]:
            if hasattr(msg, "tool_calls"):
                for tc in msg.tool_calls:
                    if tc["name"] == "market_trading_tool":
                        args = tc["args"]
                        assert args.get("symbol", "").upper() == "TSLA"
                        assert abs(float(args.get("capital", 0)) - 7500.0) < 1.0
```

4.3 Retry Logic Tests

```
# test_retry.py
import time
import pytest
from unittest.mock import MagicMock

class TestCallWithRetry:
```

```python
    def test_succeeds_on_first_attempt(self):
        func = MagicMock(return_value="ok")
        result = call_with_retry(func, retries=3, delay=0)
        assert result == "ok"
        assert func.call_count == 1

    def test_retries_on_failure_then_succeeds(self):
        func = MagicMock(side_effect=[Exception("fail"), Exception("fail"), "ok"])
        result = call_with_retry(func, retries=3, delay=0)
        assert result == "ok"
        assert func.call_count == 3

    def test_raises_after_max_retries_exceeded(self):
        func = MagicMock(side_effect=Exception("always fails"))
        with pytest.raises(Exception, match="Max retries exceeded"):
            call_with_retry(func, retries=3, delay=0)
        assert func.call_count == 3

    def test_delay_between_retries(self):
        func = MagicMock(side_effect=[Exception("fail"), "ok"])
        start = time.time()
        call_with_retry(func, retries=3, delay=0.1)
        elapsed = time.time() - start
        assert elapsed >= 0.1, "Should sleep at least once between retries"
```

---

5. Summary

Area	Status	Notes
External tools identified	☑	BLS API, Alpha Vantage, LangChain, OpenAI
Function calling schemas	☑	Google-style docstrings parsed via `@tool`
Error handling	☑	Try/except in all tools + API response guards
Retry logic	☑	`call_with_retry` with configurable attempts/delay
Tool selection tests	➕ Added	Agent-level tests for correct tool routing
Parameter correctness tests	➕ Added	Unit tests verifying extracted args match user intent
Retry logic tests	➕ Added	Tests for success, partial failure, and full failure cases